Orbit Camera: Euler-to-Quaternion Refactor

engvis

September 2, 2026

1 Problem Statement

The orbit camera in `crates/engvis-core/src/camera.rs` originally stored its orientation as two Euler angles:

- `yaw` — azimuth around the world Y axis,
- `pitch` — elevation, clamped to $\pm(\frac{\pi}{2} - 0.01) \approx \pm 89.4°$.

Coordinate convention. The camera now uses an engineering Z-up convention (XY ground plane, $+Z$ upward), matching common CAD/CAE coordinate systems. All preset views and the default orientation rotate around the world Z axis for yaw, not Y.

When the user drags the mouse to rotate the view vertically, the camera stops responding once `pitch` reaches the clamp boundary. The view cannot be rotated past nearly straight up/down, which is perceived as a hard boundary on interaction.

2 Root Cause Analysis

2.1 Pitch Clamp

The clamp is required by the Euler representation itself, not by any physical constraint of the scene. In `orbit()`:

```
pub fn orbit(&mut self, delta_yaw: f32, delta_pitch: f32) {
    self.yaw += delta_yaw;
    self.pitch = (self.pitch + delta_pitch).clamp(
        -std::f32::consts::FRAC_PI_2 + 0.01,
         std::f32::consts::FRAC_PI_2 - 0.01,
    );
}
```

Without the clamp, `pitch` would exceed $\pm\frac{\pi}{2}$, which causes two failure modes:

1. **Gimbal lock.** At `pitch` $= \pm\frac{\pi}{2}$, the yaw and pitch rotation axes coincide, and the camera loses one degree of freedom. Subsequent yaw input produces no visible rotation.
2. **Up-vector flip.** `view_matrix()` uses `Mat4::look_at_rh(pos, target, Vec3::Y)`, which degenerates when the view direction aligns with $\pm Y$: the cross product forward $\times$ up vanishes, producing a NaN view matrix.

2.2 Why Euler Angles Are Inappropriate

Euler angles parameterise SO(3) with a triple (α, β, γ), but the map

$$(\alpha, \beta, \gamma) \mapsto R_z(\alpha) R_y(\beta) R_z(\gamma)$$

is singular at $\beta = \pm\frac{\pi}{2}$ (gimbal lock). Any clamping strategy therefore imposes an artificial boundary on user input, even though the underlying rotation space $\mathrm{SO}(3) \cong \mathbb{RP}^3$ has no such boundary.

3 Solution: Quaternion Orientation

3.1 Representation

Replace the two Euler angles with a single unit quaternion `orientation: Quat` that maps camera-local axes to world-space directions:

- camera $+X$ (right) $\mapsto$ `orientation * Vec3::X`,
- camera $+Y$ (up) $\mapsto$ `orientation * Vec3::Y`,
- camera $+Z$ (back, away from target) $\mapsto$ `orientation * Vec3::Z`.

The camera position is then

$$\mathbf{p}_{\text{cam}} = \mathbf{t} + \mathbf{q}\,(d \cdot \hat{\mathbf{z}}),$$

where $\mathbf{t}$ is the orbit target, $\mathbf{q}$ the orientation quaternion, and d the orbit distance.

```rust
pub struct OrbitCamera {
    pub target: Vec3,
    pub orientation: Quat,
    pub distance: f32,
    pub fov_y: f32,
    pub near: f32,
    pub far: f32,
    pub aspect_ratio: f32,
}

pub fn position(&self) -> Vec3 {
    self.target + self.orientation * (Vec3::Z * self.distance)
}
```

3.2 Incremental Orbit Rotation

Mouse drag produces two incremental rotations per frame, both applied in the camera's **local space** (post-multiplication):

- **Yaw** $\Delta\alpha$ — rotation about the camera's own up axis Y. Applied as a post-multiplication $\mathbf{q}' = \mathbf{q}\, R_Y(\Delta\alpha)$ so the view spins left/right consistently with the screen drag, *regardless of the camera's current orientation.*
- **Pitch** $\Delta\beta$ — rotation about the camera's local X axis. Applied as a post-multiplication $\mathbf{q}' \leftarrow \mathbf{q}'\, R_X(-\Delta\beta)$, where the sign preserves the "drag up = look up" convention.

$$\mathbf{q}_{\text{new}} = \text{normalize}(\mathbf{q}_{\text{old}} \cdot R_Y(\Delta\alpha) \cdot R_X(-\Delta\beta)).$$

Why local-space yaw, not world-Y. The original implementation pre-multiplied the yaw rotation about the *world* Y axis ($R_Y\,\mathbf{q}$) to keep the horizon level. However, once the camera is rolled upside-down (e.g. after pitching past 90°), the world-Y yaw rotates in the *opposite* direction to the screen drag: dragging left turns the view right. Switching to local-space yaw ($\mathbf{q}\,R_Y$) makes the on-screen response always match the drag direction, at the cost of the horizon no longer staying level after a roll—an acceptable trade-off for an interactive inspection camera.

Because quaternion multiplication is a closed, smooth operation on SO(3), no clamp is required and no singularity arises at any orientation.

```
pub fn orbit(&mut self, drag_x: f32, drag_y: f32) {
    // Horizontal drag -> yaw about the camera's own up axis.
    // Vertical drag   -> pitch about the camera's own right axis.
    // Negate the vertical term so dragging up raises the camera.
    let yaw_rot   = Quat::from_rotation_y(drag_x);
    let pitch_rot = Quat::from_rotation_x(-drag_y);
    self.orientation = (self.orientation * yaw_rot * pitch_rot).normalize();
}
```

The parameter names `drag_x`/`drag_y` (rather than `delta_yaw`/`delta_pitch`) reflect that the inputs are screen-space drag deltas, not angular increments—the conversion to rotation angle happens inside `orbit` via the caller-supplied sensitivity factor.

The final `.normalize()` guards against floating-point drift after many incremental updates; it is a numerical safeguard, not a behavioural clamp.

3.3 View Matrix from Quaternion

The previous implementation used `Mat4::look_at_rh(position(), target, Vec3::Y)`, which degenerates when forward ∥ up. The new implementation builds the view matrix directly from the orientation quaternion:

$$V = R(\mathbf{q}^{-1})\,T(-\mathbf{p}_{\text{cam}}),$$

where $R(\mathbf{q}^{-1})$ is the world-to-camera rotation and $T(-\mathbf{p}_{\text{cam}})$ translates the camera to the origin.

```
pub fn view_matrix(&self) -> Mat4 {
    let rot   = Mat4::from_quat(self.orientation.inverse());
    let trans = Mat4::from_translation(-self.position());
    rot * trans
}
```

This formulation is well-defined for every orientation, including straight up/down and upside-down views.

3.4 Preset Views

Each preset constructs the orientation quaternion directly from the relevant axis rotations:

Preset	Orientation	Description (Z-up)
`view_front`	$\mathbf{q} = R_X(+\frac{\pi}{2})$	camera at $-Y$, looking at $+Y$, Z up
`view_top`	$\mathbf{q} = I$	camera at $+Z$, looking at $-Z$, Y screen-up
`view_right`	$\mathbf{q} = R_Z(-\frac{\pi}{2})\,R_X(-\frac{\pi}{2})$	camera at $+X$, looking at $-X$, Z up
`view_iso`	$\mathbf{q} = R_Z(-0.785)\,R_X(-0.615)$	isometric 3/4 view

```rust
pub fn view_front(&mut self) {
    self.orientation = Quat::from_rotation_x(std::f32::consts::FRAC_PI_2);
}
pub fn view_top(&mut self) {
    self.orientation = Quat::IDENTITY;
}
pub fn view_right(&mut self) {
    self.orientation = Quat::from_rotation_z(-std::f32::consts::FRAC_PI_2)
        * Quat::from_rotation_x(-std::f32::consts::FRAC_PI_2);
}
pub fn view_iso(&mut self) {
    self.orientation = Quat::from_rotation_z(-0.785) * Quat::from_rotation_x
        (-0.615);
}
```

3.5 Compatibility Helper

Code that previously set `yaw`/`pitch` directly (e.g. the `colorbar3d` example's reset-view button) now uses `set_orientation_yaw_pitch`, which composes the same $R_Z \cdot R_X$ rotation:

```rust
pub fn set_orientation_yaw_pitch(&mut self, yaw: f32, pitch: f32) {
    self.orientation = Quat::from_rotation_z(yaw) * Quat::from_rotation_x(-pitch);
}
```

4 Comparison

Aspect	Euler (yaw, pitch)	Quaternion
Representation	(α, β), $\beta \in (-\frac{\pi}{2}, \frac{\pi}{2})$	unit $\mathbf{q} \in S^3$
Singularity	at $\beta = \pm\frac{\pi}{2}$ (gimbal lock)	none
Pitch clamp	required	not required
Vertical rotation	stops at $\pm 89.4°$	unlimited
View matrix	`look_at_rh` (degenerate at poles)	direct from $\mathbf{q}^{-1}$
Up vector	implicit $(+Y)$	explicit (`orientation * Vec3::Y`)
Preset views	set two scalars	construct quaternion
Interpolation	Euler interpolation flawed	SLERP / NLERP natural
Memory	8 bytes	16 bytes

5 Pipeline Summary

The complete camera update pipeline per frame:

1. **Input** (`input.rs`) — mouse drag deltas $(\Delta x, \Delta y)$ scaled by a sensitivity factor.
2. **Orbit** (left mouse) — `camera.orbit(`$-\Delta x \cdot s$`, `$-\Delta y \cdot s$`)` composes incremental local-space quaternion rotations.
3. **Pan** (right mouse) — translates `target` along camera-local X and Y. Screen y grows downward, so the drag delta is negated: `pan(`$-\Delta x \cdot s$`, `$-\Delta y \cdot s$`)` makes the object follow the cursor (drag up → object up, drag right → object right).

4. **Zoom** (scroll) — scales `distance` with a scene-radius-relative clamp $[0.02\,r,\ 500\,r]$ and resynchronises the near/far clip planes (Section 6).

5. **View matrix** — built from $\mathbf{q}^{-1}$ and $-\mathbf{p}_{\text{cam}}$.

6. **Projection** — perspective RH from `fov_y`, `aspect_ratio`, `near`, `far`.

6 Zoom-Synchronised Clip Planes

Initially `near` / `far` were computed once in `fit_to_aabb` and never touched again, while `zoom()` only changed `distance` (clamped to the fixed range $[0.1, 500]$). Consequently the object vanished whenever

$$d < \texttt{near} \quad \text{(zoom in)} \qquad \text{or} \qquad d > \texttt{far} \quad \text{(zoom out)},$$

with d the camera distance — the reported "object disappears at extreme zoom" bug.

The fix introduces a `scene_radius` r (half the fitted AABB diagonal, stored by `fit_to_aabb`). Every zoom step recomputes the clip planes from the new distance:

$$d \leftarrow \text{clamp}\big(d\,(1-\delta),\ 0.02\,r,\ 500\,r\big), \tag{1}$$

$$\texttt{near} \leftarrow \max\big(d - 2r,\ 10^{-3}d,\ 10^{-4}\big), \tag{2}$$

$$\texttt{far} \leftarrow \max\big(d + 2r,\ 10\big). \tag{3}$$

The nearest possible surface point is $d - r$ (bounding sphere), so $\texttt{near} = d - 2r$ keeps a full-radius margin in front of the geometry — robust against a slightly underestimated `scene_radius` (e.g. `colorbar3d` sets `distance` manually and never fits). When the camera moves inside the bounding sphere ($d - 2r < 0$) the near plane falls back to a small fraction of d, which stays in front of the camera. Because both planes follow d, the ratio `far`/`near` stays bounded and depth precision is preserved across the whole zoom range.

A regression test (`zoom_keeps_object_inside_clip_planes`) zooms across the reachable range for radii $\{0.05, 1, 1.73, 40\}$ and asserts the bounding sphere never leaves $[\texttt{near}, \texttt{far}]$.

7 Result

After the refactor:

- The camera rotates freely in all directions; no boundary is encountered when looking straight up or down.
- All preset views (front, top, right, iso) reproduce the previous camera framing.
- The `colorbar3d` example's reset-view button restores the same isometric orientation as before.
- Zooming in or out to any reachable distance keeps the object inside the frustum (Section 6); the previously reported disappearance at extreme zoom is fixed and covered by a unit test.
- `cargo build`, `cargo test -workspace`, and runtime startup all pass without errors.